

TECHNICAL
SAFETY &
GOVERNANCE
LAB

OxML Summer School
2026



Probing Neural Networks

Fazl Barez

✕ @FazlBarez | 🌐 fazlbarez.com

Today's structure

PART I	Motivation	Why look inside? Debugging, shortcuts, safety, probes now shipping in production.
PART II	Linear Hypothesis	What representations are. Notation. Why linearity emerges. When it breaks.
PART III	Probe Types	Linear · Nonlinear · Structural · Causal, what each proves.
PART IV	Evaluation	The three baselines. Selectivity. Why raw accuracy misleads.
PART V	Findings	What probing has discovered about BERT, GPT, and safety.
PART VI	Limits	Causation gap. The probe-as-new-model problem. Superposition.
PART VII	Safety	Deployed probes. What representations reveal that outputs cannot.
PART VIII	Designing a Study	From question to defensible conclusion.

PART I

Motivation

Why look inside neural networks?

You can't fix what you can't see

Large models achieve impressive benchmark scores, and then fail in deployment in ways no benchmark predicted.

*“High test accuracy tells you a model works on average.
It tells you almost nothing about what it learned.”*

To debug, audit, or certify a model, we need to see what it encoded, not just what it says. That gap between behaviour and representation is exactly what makes looking inside so productive.

A canonical failure: skin cancer

A classifier trained to detect malignant skin lesions matched dermatologist-level accuracy on the test set.

A canonical failure: skin cancer

A classifier trained to detect malignant skin lesions matched dermatologist-level accuracy on the test set.

What it actually learned

The model learned to detect *rulers* in photographs, not cancer features.

Malignant lesions are more often photographed beside a measuring ruler in clinical settings. The model learned that spurious correlation perfectly. The test set had the same bias.

Standard evaluation never catches this. You need to look inside.

Three reasons to look inside

Debugging

When a model fails you need a diagnosis, not a guess. Which layer? Which feature? Was it the intended signal or an accidental one?

Three reasons to look inside

Debugging

When a model fails you need a diagnosis, not a guess. Which layer? Which feature? Was it the intended signal or an accidental one?

Spurious shortcuts

Models learn whatever predicts the training labels. Probing reveals whether they learned the right signal or an accidental one.

Three reasons to look inside

Debugging

When a model fails you need a diagnosis, not a guess. Which layer? Which feature? Was it the intended signal or an accidental one?

Spurious shortcuts

Models learn whatever predicts the training labels. Probing reveals whether they learned the right signal or an accidental one.

Safety, arguably the one that matters most

A safety-trained model may refuse harmful outputs, but the knowledge can still sit there internally, recoverable with the right adversarial prompt. Probing tests for it directly, independently of what the model says.

From research tool to deployed safeguard

Cheap

One dot product per token: $s = w \cdot h$ vs a full forward pass for an LLM judge. In production, a probe + LLM cascade beats either alone at $\sim 1/50$ th the judge's cost (Kramár et al., 2026).

Early

The probe reads h during generation, before the harmful completion exists. Output filters can only react to text already produced.

Hard to game from outside

A model (or attacker) can shape its text. Shaping mid-layer activations from the input side is a much harder target, the representation often betrays what the output conceals.

From research tool to deployed safeguard

Cheap

One dot product per token: $s = w \cdot h$ vs a full forward pass for an LLM judge. In production, a probe + LLM cascade beats either alone at $\sim 1/50$ th the judge's cost (Kramár et al., 2026).

Early

The probe reads h during generation, before the harmful completion exists. Output filters can only react to text already produced.

Hard to game from outside

A model (or attacker) can shape its text. Shaping mid-layer activations from the input side is a much harder target, the representation often betrays what the output conceals.

A taxonomy point people conflate

Not every safety classifier is a probe. Constitutional classifiers (Sharma et al., 2025) are fine-tuned LLMs reading *text*; a probe reads *activations*, different channel, different cost, different failure modes, which is why the stack wants both.

The rest of this lecture is that method, and its honest limits.

What probing is

Definition

Train a small supervised classifier on frozen hidden states to predict a property of the input.

If it succeeds, infer the property is encoded in the representation.

Frozen = the model never updates. Only the probe learns. This is what makes probing interpretable rather than fine-tuning.

Where it sits

Between output-only behavioural testing and deep mechanistic circuit tracing.

Probing in the interpretability toolkit

Method	Question	Approach	Level
Probing (today)	What is encoded in h ?	Train a classifier on frozen representations.	Representations
Saliency / Attribution	Which input features caused this output?	Gradient-based methods.	Input features
Mechanistic interp.	Which circuits implement a behaviour?	Trace attention heads and MLPs.	Circuits
Behavioural testing	What does the model do across inputs?	No internal access.	Outputs

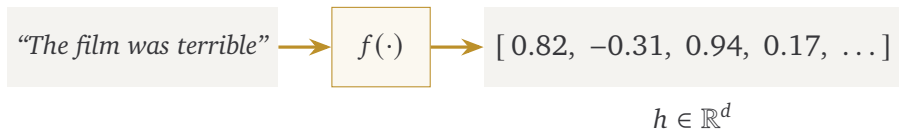
PART II

The Linear Hypothesis

The foundational assumption, and when it holds

What a representation is

For every input, the network produces a vector h at each layer, a point in d -dimensional space.



$d = 768$ for BERT-base

$d = 12,288$ for GPT-3

Everything the model knows about this input is compressed into h .

Notation for the rest of the talk

$$x, y \in \mathcal{Y}$$

input; property label (e.g. $\mathcal{Y} = \{\text{pos}, \text{neg}\}$)

$$h^{(\ell)} = f^{(\ell)}(x) \in \mathbb{R}^d$$

hidden state at layer ℓ , encoder f **frozen** throughout

$$w \cdot h = w^\top h$$

dot product, the only operation a linear probe performs

$$\|h\|, \hat{w} = w/\|w\|$$

norm; unit direction

$$\text{proj}_w(h) = (\hat{w} \cdot h) \hat{w}$$

the component of h along direction w

$$g_\theta : \mathbb{R}^d \rightarrow \Delta(\mathcal{Y})$$

the probe, hidden states in, label probabilities out

$$\sigma(z), \text{softmax}$$

squash scores into probabilities

One sentence to remember: *a concept direction w turns geometry into a score, $s = w \cdot h$, and a threshold on s into a classifier.*

Three tools, one line each

1. Geometry (Parts II–III: what probes test)

A direction w and bias b define a hyperplane $\{h : w \cdot h + b = 0\}$. Linear probing asks: does a hyperplane separate the concept?

2. Optimisation (Parts III–IV: how probes are trained and judged)

$\hat{\theta} = \arg \min_{\theta} \frac{1}{n} \sum_i \ell(g_{\theta}(h_i), y_i)$. Fit on training pairs, report on held-out data, a probe is judged like any classifier: **against baselines**.

3. Information (Part VI: what probing can even mean)

Entropy $H(Y)$ = uncertainty in the labels. Mutual information $I(H; Y)$ = how much h reduces it. Data-processing inequality: no function of h can create information about Y that h does not already carry, the one-line theorem behind the deepest critique of probing.

The linear representation hypothesis

Concepts correspond to *directions* in $\mathbb{R}^d$.

The linear representation hypothesis

Concepts correspond to *directions* in $\mathbb{R}^d$.

Not clusters. Not nonlinear manifolds. Directions.

What it commits you to

Moving along direction d_c in $\mathbb{R}^d$ changes concept c while leaving others unchanged. Read that twice, it is an *intervention* claim, not just a geometry claim. It returns in Part VI.

The clearest evidence: word analogies

$$\text{king} - \text{man} + \text{woman} = \text{queen}$$

Mikolov et al. (2013), arithmetic in word embedding space

Gender is encoded as a direction d_{gender} in $\mathbb{R}^d$. Subtracting d_{gender} from “king” leaves royalty without gender, which is “queen”.

If the hypothesis holds, a single dot product detects any linearly encoded concept.

Why training pressures representations toward linearity

$$\text{logits} = W_{\text{out}} \cdot h$$

Why training pressures representations toward linearity

$$\text{logits} = W_{\text{out}} \cdot h$$

class scores the model outputs

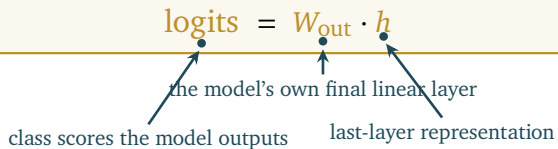
Why training pressures representations toward linearity

$$\text{logits} = W_{\text{out}} \cdot h$$

the model's own final linear layer

class scores the model outputs

Why training pressures representations toward linearity



The diagram shows the equation $\text{logits} = W_{\text{out}} \cdot h$ inside a light yellow rectangular box. Below the box, three blue arrows point to the terms in the equation: one from 'logits' to 'class scores the model outputs', one from ' W_{out} ' to 'the model's own final linear layer', and one from ' h ' to 'last-layer representation'.

$$\text{logits} = W_{\text{out}} \cdot h$$

class scores the model outputs the model's own final linear layer last-layer representation

Why training pressures representations toward linearity

The diagram shows the equation $\text{logits} = W_{\text{out}} \cdot h$ inside a light yellow rectangular box. Below the box, three blue arrows point to the terms in the equation: one from the text 'class scores the model outputs' to 'logits', one from 'the model's own final linear layer' to ' W_{out} ', and one from 'last-layer representation' to ' h '.

$$\text{logits} = W_{\text{out}} \cdot h$$

class scores the model outputs the model's own final linear layer last-layer representation

For class c to score high, h must project strongly onto row c of W_{out} , gradient descent enforces this at the output.

Key implication

A linear output head makes linear separability at the *final* layer a training objective by construction. Upstream layers feel this only indirectly, the linear tendency there is an empirical regularity, not a guarantee. That is why probing is an experiment, not a derivation.

When the hypothesis holds: and when it breaks

Holds well for

- Simple binary or categorical properties
- Frequent factual associations
- Properties supervised to output linearly
- POS tags, sentiment, named entities

When the hypothesis holds: and when it breaks

Holds well for

- Simple binary or categorical properties
- Frequent factual associations
- Properties supervised to output linearly
- POS tags, sentiment, named entities

Breaks down for

- Complex compositional structure
- Rare or unseen concepts
- Abstract goal representations
- Properties absent from training signal
- Features stored in superposition

Remember

The hypothesis is an empirical regularity, not a law. Probing tests whether it holds for your specific concept.

PART III

Probe Types

Linear · Nonlinear · Structural · Causal, what each tells you

The probing setup: as a learning problem

$$\hat{\theta} = \operatorname{argmin}_{\theta} \frac{1}{n} \sum_i \ell(g_{\theta}(h_i), y_i)$$

The probing setup: as a learning problem

$$\hat{\theta} = \operatorname{argmin}_{\theta} \frac{1}{n} \sum_i \ell(g_{\theta}(h_i), y_i)$$

the trained probe,
the only part that learns

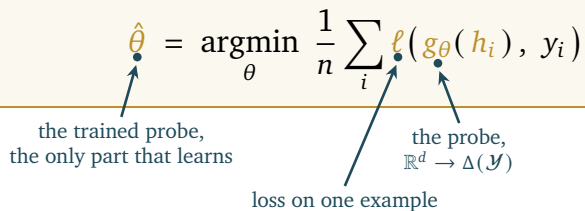
The probing setup: as a learning problem

$$\hat{\theta} = \operatorname{argmin}_{\theta} \frac{1}{n} \sum_i \ell(g_{\theta}(h_i), y_i)$$

the trained probe,
the only part that learns

loss on one example

The probing setup: as a learning problem



The diagram illustrates the probing setup as a learning problem. It features a central equation within a light yellow rectangular box, bordered by a thin orange line. The equation is $\hat{\theta} = \operatorname{argmin}_{\theta} \frac{1}{n} \sum_i \ell(g_{\theta}(h_i), y_i)$. Three blue arrows point from text labels below to specific parts of the equation: one points to $\hat{\theta}$, another to ℓ , and a third to g_{θ} . The labels are: 'the trained probe, the only part that learns' (pointing to $\hat{\theta}$), 'loss on one example' (pointing to ℓ), and 'the probe, $\mathbb{R}^d \rightarrow \Delta(\mathcal{Y})$ ' (pointing to g_{θ}). The symbols $\hat{\theta}$, ℓ , and h_i in the equation are highlighted in orange.

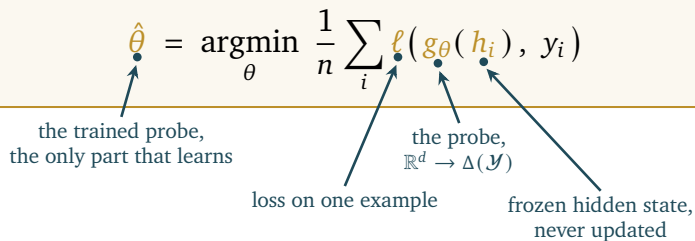
$$\hat{\theta} = \operatorname{argmin}_{\theta} \frac{1}{n} \sum_i \ell(g_{\theta}(h_i), y_i)$$

the trained probe,
the only part that learns

loss on one example

the probe,
 $\mathbb{R}^d \rightarrow \Delta(\mathcal{Y})$

The probing setup: as a learning problem



The diagram illustrates the probing setup as a learning problem. It features a central equation within a light yellow rectangular box, with four blue arrows pointing from descriptive text labels below to specific parts of the equation. The equation is:
$$\hat{\theta} = \operatorname{argmin}_{\theta} \frac{1}{n} \sum_i \ell(g_{\theta}(h_i), y_i)$$
 The labels and their targets are: 1. "the trained probe, the only part that learns" points to $\hat{\theta}$. 2. "loss on one example" points to ℓ . 3. "the probe, $\mathbb{R}^d \rightarrow \Delta(\mathcal{Y})$ " points to g_{θ} . 4. "frozen hidden state, never updated" points to h_i .

$$\hat{\theta} = \operatorname{argmin}_{\theta} \frac{1}{n} \sum_i \ell(g_{\theta}(h_i), y_i)$$

the trained probe,
the only part that learns

loss on one example

the probe,
 $\mathbb{R}^d \rightarrow \Delta(\mathcal{Y})$

frozen hidden state,
never updated

The probing setup: as a learning problem

The diagram shows the equation $\hat{\theta} = \operatorname{argmin}_{\theta} \frac{1}{n} \sum_i \ell(g_{\theta}(h_i), y_i)$ inside a yellow box. Four blue arrows point from text labels below to specific parts of the equation:

- From $\hat{\theta}$ to "the trained probe, the only part that learns"
- From the first ℓ to "loss on one example"
- From g_{θ} to "the probe, $\mathbb{R}^d \rightarrow \Delta(\mathcal{Y})$ "
- From h_i to "frozen hidden state, never updated"

$$\hat{\theta} = \operatorname{argmin}_{\theta} \frac{1}{n} \sum_i \ell(g_{\theta}(h_i), y_i)$$

the trained probe,
the only part that learns

loss on one example

the probe,
 $\mathbb{R}^d \rightarrow \Delta(\mathcal{Y})$

frozen hidden state,
never updated

Report: held-out accuracy, **against baselines** (Part IV). A probe is judged like any classifier.

The one non-negotiable

The encoder f is never updated. Without this you are fine-tuning, not probing.

Four types of probe

1 Linear probe	Logistic regression on h .	Tests for linear decodability.
2 Nonlinear (MLP)	Multi-layer network on h .	Tests decodability without the linearity constraint.
3 Structural probe	Learns a metric over h .	Tests geometric structure (e.g. parse trees).
4 Causal (patching)	Activation patching.	Tests whether h is actually used for outputs.

Type 1: the linear probe

$$\hat{y} = \text{softmax}(W \cdot h + b)$$

Type 1: the linear probe

$$\hat{y} = \text{softmax}(W \cdot h + b)$$

↑
probabilities over labels

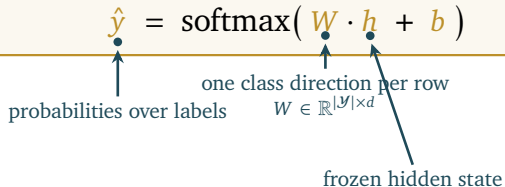
Type 1: the linear probe

$$\hat{y} = \text{softmax}(W \cdot h + b)$$

probabilities over labels

one class direction per row
 $W \in \mathbb{R}^{|\mathcal{Y}| \times d}$

Type 1: the linear probe



The diagram shows the equation $\hat{y} = \text{softmax}(W \cdot h + b)$ inside a light yellow rectangular box. Below the box, three blue arrows point to specific parts of the equation: one to $\hat{y}$, one to W , and one to h . The arrow to $\hat{y}$ is labeled "probabilities over labels". The arrow to W is labeled "one class direction per row" and " $W \in \mathbb{R}^{|\mathcal{Y}| \times d}$ ". The arrow to h is labeled "frozen hidden state".

$$\hat{y} = \text{softmax}(W \cdot h + b)$$

probabilities over labels

one class direction per row
 $W \in \mathbb{R}^{|\mathcal{Y}| \times d}$

frozen hidden state

Type 1: the linear probe

The diagram shows the equation $\hat{y} = \text{softmax}(W \cdot h + b)$ inside a light yellow rectangular box. Below the box, four blue arrows point to specific parts of the equation: one to $\hat{y}$, one to W , one to h , and one to b . Each arrow is accompanied by a text label. The label for W also includes the matrix dimension $W \in \mathbb{R}^{|\mathcal{Y}| \times d}$.

$$\hat{y} = \text{softmax}(W \cdot h + b)$$

probabilities over labels

one class direction per row
 $W \in \mathbb{R}^{|\mathcal{Y}| \times d}$

frozen hidden state

bias

Type 1: the linear probe

The diagram shows the equation $\hat{y} = \text{softmax}(W \cdot h + b)$ inside a light yellow box. Below the box, several annotations with arrows point to parts of the equation: an arrow from $\hat{y}$ points to the text 'probabilities over labels'; an arrow from W points to the text 'one class direction per row' and $W \in \mathbb{R}^{|\mathcal{Y}| \times d}$; an arrow from h points to the text 'frozen hidden state'; and an arrow from b points to the text 'bias'.

$$\hat{y} = \text{softmax}(W \cdot h + b)$$

probabilities over labels

one class direction per row
 $W \in \mathbb{R}^{|\mathcal{Y}| \times d}$

frozen hidden state

bias

What success proves

The concept is linearly encoded as a direction in $\mathbb{R}^d$. Each row of W is a class direction, $W^T[c]$ is the concept direction, usable for steering and editing.

What it does NOT prove

That the model uses this representation for its outputs. Information is accessible, not necessarily used.

Type 2: the nonlinear (MLP) probe

$$\hat{y} = \text{softmax}(w_2 \cdot \text{ReLU}(W_1 \cdot h + b_1) + b_2)$$

Type 2: the nonlinear (MLP) probe

$$\hat{y} = \text{softmax}(w_2 \cdot \text{ReLU}(W_1 \cdot h + b_1) + b_2)$$

↑
probe's own layers,
capacity lives here

Type 2: the nonlinear (MLP) probe

$$\hat{y} = \text{softmax}(w_2 \cdot \text{ReLU}(W_1 \cdot h + b_1) + b_2)$$

probe's own layers
capacity lives here

nonlinearity, exactly what
the linear probe lacks

Type 2: the nonlinear (MLP) probe

$$\hat{y} = \text{softmax}(w_2 \cdot \text{ReLU}(W_1 \cdot h + b_1) + b_2)$$

probe's own layers
capacity lives here

still frozen

nonlinearity, exactly what
the linear probe lacks

Type 2: the nonlinear (MLP) probe

$$\hat{y} = \text{softmax}(w_2 \cdot \text{ReLU}(W_1 \cdot h + b_1) + b_2)$$

probe's own layers
capacity lives here

still frozen

nonlinearity, exactly what
the linear probe lacks

Advantage

Can detect properties not linearly encoded, features in superposition, nonlinear combinations. Useful when the linear probe fails but you want to confirm information is present.

Danger

High accuracy is hard to interpret. The MLP may solve the task itself rather than reading h . You lose the mechanistic claim. Every step up in complexity requires explicit justification.

The fundamental difference

Linear probe success

Concept is a **direction** in $\mathbb{R}^d$. (Mechanistically interpretable, the strongest claim.)

The fundamental difference

Linear probe success

Concept is a **direction** in $\mathbb{R}^d$. (Mechanistically interpretable, the strongest claim.)

MLP probe success

Concept is **decodable** from h . (Weaker, decodability, not linearity.)

These are very different claims

One asserts geometric structure in the representation. The other says information is not entirely absent. Know which you are making.

The probe complexity spectrum

Logistic regression

One linear layer. Highest selectivity. Strongest claims. Always start here.

Shallow MLP ($d = 10-50$)

Nonlinear, low capacity. Can detect some superposed features. Weaker claims.

Deep MLP ($d = 1000+$)

Very expressive. May solve the task itself. Avoid for interpretive claims.

Rule

Use the simplest probe that answers your question. L2 regularisation does *not* improve selectivity, only capacity reduction does. Why: the claim's strength comes from restricting the probe family $\mathcal{V}$ (Part VI). Weight decay shrinks weights within the family; it does not shrink the family.

Type 3: the structural probe

$$d(i, j)^2 = \| B h_i - B h_j \|^2$$

Type 3: the structural probe

$$d(i, j)^2 = \| B h_i - B h_j \|^2$$

distance between words i, j
in the parse tree



Type 3: the structural probe

$$d(i, j)^2 = \| B h_i - B h_j \|^2$$

distance between words i, j
in the parse tree

learned low-rank map,
 $k \times d$, with $k \ll d$

Type 3: the structural probe

$$d(i, j)^2 = \| B h_i - B h_j \|^2$$

distance between words i, j
in the parse tree

learned low-rank map,
 $k \times d$, with $k \ll d$

Trained by $\min_B \sum (d_T(i, j) - \|B(h_i - h_j)\|)^2$, the low rank k is itself a finding.

What it found

Does a linear transformation of h preserve parse-tree topology? For BERT: yes, without any syntactic supervision. Structural probes test metric geometry, not just class boundaries.

The gap every other probe leaves open

What linear, MLP, and structural probes show
Information is **accessible** from h .

What they do not show
That the model actually **uses** h for its outputs.
Accessibility is not the same as causal relevance.

Activation patching: the procedure

Step 1 · Run clean

Step 1: Run clean x_1

Record the hidden state h_L at layer L .

Activation patching: the procedure

Step 1 · Run clean

Step 2 · Run corrupt

Step 1: Run clean x_1

Record the hidden state h_L at layer L .

Step 2: Run corrupted x_2

Change the input in a controlled way. Record $h_L(\text{corrupt})$ at layer L .

Activation patching: the procedure

Step 1 · Run clean

Step 2 · Run corrupt

Step 3 · Patch h

Step 4 · Observe

Step 1: Run clean x_1

Record the hidden state h_L at layer L .

Step 2: Run corrupted x_2

Change the input in a controlled way. Record $h_L(\text{corrupt})$ at layer L .

Step 3: Patch

Rerun clean, swap in $h_L(\text{corrupt})$ at layer L only.

Step 4: Observe

Output shifts $\Rightarrow h_L$ causally used. No shift $\Rightarrow$ not used from here.

Activation patching: the causal claim

$$\text{effect} = \Delta \text{logit}(y^*) \quad \text{under} \quad do(h^{(L)} \leftarrow h_{\text{corrupt}}^{(L)})$$

Activation patching: the causal claim

$$\text{effect} = \Delta \text{logit}(y^*) \quad \text{under} \quad do(h^{(L)} \leftarrow h_{\text{corrupt}}^{(L)})$$

↑
shift on the answer token

Activation patching: the causal claim

$$\text{effect} = \Delta \text{logit}(y^*) \quad \text{under} \quad do(h^{(L)} \leftarrow h_{\text{corrupt}}^{(L)})$$

↑
shift on the answer token

↑
set by intervention, not observation

Activation patching: the causal claim

$$\text{effect} = \Delta \text{logit}(y^*) \quad \text{under} \quad do(h^{(L)} \leftarrow h_{\text{corrupt}}^{(L)})$$

↑
shift on the answer token

↑
set by intervention, not observation

This is the only probe type that establishes causation.

Comparing the four probe types

Probe type	What success proves	Linearity	Causation
Linear probe	Direction in $\mathbb{R}^d$	Yes	No
MLP / nonlinear	Decodable from h	No	No
Structural probe	Metric structure in $\mathbb{R}^d$	Yes	No
Causal (patching)	h is causally used	No	Yes

Rule

Always start with the linear probe. Justify any increase in complexity explicitly.

PART IV

Evaluation

Why raw accuracy is almost meaningless alone

The core problem

85% accuracy.

The core problem

85% accuracy.

Is this good?

The core problem

85% accuracy.

Is this good?

You cannot know without baselines. 85% might be extraordinary, or worse than chance.

Baseline 1: majority class

Always predict the most common label

Requires zero learning and zero information from representations.
If your probe barely beats this, it has learned almost nothing.

Example

80% of examples are positive $\Rightarrow$ majority baseline = 80%.

A probe at 82% adds only 2 pp beyond ignoring representations entirely.

Baseline 2: random representation

Same probe, same labels, h from a randomly initialised, untrained encoder

Same architecture, same tokenisation, no training. (Not a random projection of trained features, that is a different, weaker control.) Isolates accuracy from data statistics alone; controls for label imbalance, lexical shortcuts, and dataset artefacts.

Interpretation

If probe on trained $h \approx$ probe on random h , training added almost no useful signal for this concept.

Selectivity (Hewitt & Liang, 2019)

$$\text{selectivity} = \text{acc}(\mathcal{D}) - \text{acc}(\tilde{\mathcal{D}})$$

Selectivity (Hewitt & Liang, 2019)

$$\text{selectivity} = \text{acc}(\mathcal{D}) - \text{acc}(\tilde{\mathcal{D}})$$

probe on the real labels



Selectivity (Hewitt & Liang, 2019)

$$\text{selectivity} = \text{acc}(\mathcal{D}) - \text{acc}(\tilde{\mathcal{D}})$$

probe on the real labels

same probe, labels randomised
per word type

Selectivity (Hewitt & Liang, 2019)

$$\text{selectivity} = \text{acc}(\mathcal{D}) - \text{acc}(\tilde{\mathcal{D}})$$

probe on the real labels

same probe, labels randomised
per word type

Control task: each word type gets a fixed random label; tokens inherit their type's label. Type-level assignment means the control is learnable *only by memorising the lexicon*, exactly the probe capacity you want to subtract. No representation-level signal can help: the labels are random.

What control accuracy measures

The probe's memorisation capacity. Nothing else.

Reading selectivity

High selectivity

Linguistic accuracy $\gg$ control accuracy.

The representations are doing work the probe alone cannot do. Strong evidence of encoding.

Low selectivity

Linguistic accuracy $\approx$ control accuracy.

The probe's memorisation explains most of the result. Weak evidence, beware over-claiming.

Hewitt & Liang (2019), Designing and Interpreting Probes with Control Tasks

Selectivity in practice: ELMo, POS tagging

Probe type	Linguistic acc.	Control acc.	Selectivity
Linear	95.8%	71.2%	24.6%
MLP (d=1000)	97.3%	93.4%	3.9%
MLP (d=10)	96.2%	77.4%	18.8%

Key lesson

MLP (d=1000): highest raw accuracy but selectivity only 3.9%. Linear probe selectivity 24.6%, the stronger result.

PART V

Findings

Key empirical results from the literature

BERT rediscovers the NLP pipeline

Layers 3–5

POS tags, morphology.
Basic syntactic structure.

Layers 6–9

Semantic roles, coreference.
Named entity recognition.

Layers 10–12

World knowledge.
Factual associations.

BERT rediscovers the classical NLP pipeline, without any linguistic supervision.

Tenney et al. (2019), BERT Rediscovers the Classical NLP Pipeline

Factual knowledge lives in late-layer key–value memories

Fill-in-the-blank (zero fine-tuning), the behavioural signature

“The capital of France is [MASK].” → Paris · “Einstein was born in [MASK].” → Germany ·
“Beethoven was a [MASK].” → composer

Note: this is behavioural evidence (outputs), the representational story is below.

Implication, the representational story

FFN layers act as key–value memories: keys match input patterns, values write concept directions into the residual stream (Geva et al., 2021). This is what licenses locating, and editing, specific facts (ROME, Meng et al., 2022). Petroni’s fill-in-the-blank results are the behavioural signature of this structure.

Models encode a truth direction

$$\text{find } w \text{ s.t. } w \cdot h(x^+) > w \cdot h(x^-)$$

Models encode a truth direction

find w s.t. $w \cdot h(x^+) > w \cdot h(x^-)$

↑
the truth direction

Models encode a truth direction

find w s.t. $w \cdot h(x^+) > w \cdot h(x^-)$

the truth direction

a statement

Models encode a truth direction

find w s.t. $w \cdot h(x^+) > w \cdot h(x^-)$

the truth direction a statement its negation

Models encode a truth direction

The diagram illustrates the optimization of a truth direction vector w . It features a light yellow rectangular box containing the mathematical expression: $\text{find } w \text{ s.t. } w \cdot h(x^+) > w \cdot h(x^-)$. Below this box, three blue arrows point upwards to specific parts of the equation: the first arrow points to the vector w and is labeled "the truth direction"; the second arrow points to the vector $h(x^+)$ and is labeled "a statement"; the third arrow points to the vector $h(x^-)$ and is labeled "its negation".

$$\text{find } w \text{ s.t. } w \cdot h(x^+) > w \cdot h(x^-)$$

the truth direction a statement its negation

No ground-truth labels needed, only the consistency constraint from statement / negation pairs.

Result: recovers a truth direction **without any labels**, outperforming the model's own zero-shot answers by ~ 4 points on average across 10 datasets. The model internally distinguishes true from false, even when its output is wrong.

Caveat: later work (Farquhar et al., 2023) shows unsupervised methods can latch onto other salient features than truth. The direction exists; its identity needs validation.

Burns et al. (2022), Discovering Latent Knowledge in Language Models Without Supervision

Probe directions can steer model behaviour

$$h' = h + \alpha \cdot w_c \quad (\text{at every layer})$$

Probe directions can steer model behaviour

$$h' = h + \alpha \cdot w_c \quad (\text{at every layer})$$



strength, $\alpha > 0$ amplifies,
 $\alpha < 0$ suppresses

Probe directions can steer model behaviour

$$h' = h + \alpha \cdot w_c \quad (\text{at every layer})$$

strength, $\alpha > 0$ amplifies,
 $\alpha < 0$ suppresses

unit-norm concept
direction from the probe

Probe directions can steer model behaviour

$$h' = h + \alpha \cdot w_c \quad (\text{at every layer})$$

strength, $\alpha > 0$ amplifies,
 $\alpha < 0$ suppresses

unit-norm concept
direction from the probe

Applications

Add truth direction $\rightarrow$ more accurate outputs.
Subtract harmful-intent direction $\rightarrow$ reduced harmful generation. No fine-tuning required.

The catch, Context Matters (Agarwal, Navani, Barez, 2025)

The interesting question is when this generalises: a direction that steers in one setting often doesn't transfer to another. Worth checking before you rely on it. NeurIPS 2025 MI Workshop.

Sparse autoencoders (SAEs)

The idea

Train a sparse autoencoder to decompose h into a higher-dimensional sparse basis. Each feature fires for one interpretable concept, monosemanticity without human-defined labels.

$$h \approx W_{\text{dec}} \cdot f(h), \quad f(h) \text{ sparse.}$$

Key finding

SAE features transfer across model families, universal representational geometry. Extends probing from human-defined labels to model-defined features.

Lan, Torr, Barez et al.: universal feature spaces.
Heindrich, Torr, Barez et al.: do SAEs generalise?
Kharlapenko et al.: scaling sparse feature circuits.

Lan, Torr, Barez et al. (2024/2025) · Kharlapenko, [...], Barez et al. ICML 2025

PART VI

Limitations

What probes cannot tell you, and how to report honestly

The fundamental limitation

Probes show information is **accessible** in h .

Not that the model **uses** h for its outputs.

These are completely different questions. A probe is an observation, it modifies nothing and does not establish that h is causally used.

The causation gap: a concrete example

A probe finds sentiment in BERT layer 6, 93% accuracy, selectivity 25%.

You ablate the sentiment direction from layer 6.

Model outputs do not change.

Explanation

Sentiment existed in layer 6 as a by-product of computation, but the model was reading it from elsewhere, or not using it for this output at all. Finding information does not mean the model was using it there.

This experiment has a name, amnesic probing (Elazar et al., 2021), and a protocol. Coming up.

The probe confounder problem

The problem

High probe accuracy can reflect the probe's expressivity, its capacity to memorise, not the representation's content.

Solution: selectivity

High selectivity means representations add signal beyond memorisation. If selectivity is low, be very cautious about any conclusions.

Two ways a probe can lie to you

Found $\neq$ used

The information is genuinely in h , but the model never reads it from there. A by-product of computation, not an input to it. (The causation gap. Fix: intervene.)

Two ways a probe can lie to you

Found $\neq$ used

The information is genuinely in h , but the model never reads it from there. A by-product of computation, not an input to it. (The causation gap. Fix: intervene.)

Found $\neq$ encoded

The information is not organised in h at all, the *probe* computed it. An expressive probe trained on (h, y) is itself a model of the task. (The readout problem. Fix: constrain the probe.)

Rule

Every probing claim must survive both objections. The next four slides make the second one precise.

The injectivity problem

$$I(H;Y) \leq I(X;Y)$$

The injectivity problem

$$I(H; Y) \leq I(X; Y)$$

what an unconstrained
probe measures



The injectivity problem

$$I(H; Y) \leq I(X; Y)$$

what an unconstrained
probe measures

data-processing inequality

The injectivity problem

$$I(H; Y) \leq I(X; Y)$$

what an unconstrained
probe measures

data-processing inequality

what the dataset
already contains

The injectivity problem

$$I(H; Y) \leq I(X; Y)$$

what an unconstrained
probe measures

data-processing inequality

what the dataset
already contains

Consequence

An unconstrained probe measures the dataset, not the representation, near-equality holds when f loses almost nothing. A sufficiently powerful probe recovers the concept from a randomly initialised encoder too, because the information was never gone.

Under an unconstrained probe family, “the concept is decodable from h ” is trivially true.

What probing actually measures: usable information

$$I_{\mathcal{V}}(H \rightarrow Y) = H_{\mathcal{V}}(Y) - H_{\mathcal{V}}(Y | H)$$

What probing actually measures: usable information

$$I_{\mathcal{V}}(H \rightarrow Y) = H_{\mathcal{V}}(Y) - H_{\mathcal{V}}(Y | H)$$

↑
restricted to probe family $\mathcal{V}$,
linear, shallow MLP, ...

What probing actually measures: usable information

$$I_{\mathcal{V}}(H \rightarrow Y) = H_{\mathcal{V}}(Y) - H_{\mathcal{V}}(Y | H)$$

↑
restricted to probe family $\mathcal{V}$,
linear, shallow MLP, ...

↑
uncertainty left when
probes in $\mathcal{V}$ read H

What probing actually measures: usable information

$$I_{\mathcal{V}}(H \rightarrow Y) = H_{\mathcal{V}}(Y) - H_{\mathcal{V}}(Y | H)$$

↑
restricted to probe family $\mathcal{V}$,
linear, shallow MLP, ...

↑
uncertainty left when
probes in $\mathcal{V}$ read H

$\mathcal{V}$ = all functions → recovers Shannon $I(H; Y)$, the vacuous case

$\mathcal{V}$ = linear maps → the linear probe's implicit claim, now exact

What changes

$I_{\mathcal{V}}$ can rise through layers though Shannon information only falls. Training does not create information, it re-formats it so simple readouts find it. **That re-formatting is the finding.**

MDL probing: accuracy and complexity in one number

$$L_{\text{online}} = \sum_t -\log_2 p_{\theta_t}(y_t | h_t)$$

MDL probing: accuracy and complexity in one number

$$L_{\text{online}} = \sum_t -\log_2 p_{\theta_t}(y_t | h_t)$$

total bits paid across
the data stream

MDL probing: accuracy and complexity in one number

$$L_{\text{online}} = \sum_t -\log_2 p_{\theta_t}(y_t | h_t)$$

total bits paid across
the data stream

probe trained on the
first $t-1$ examples

MDL probing: accuracy and complexity in one number

$$L_{\text{online}} = \sum_t -\log_2 p_{\theta_t}(y_t | h_t)$$

total bits paid across
the data stream

probe trained on the
first $t-1$ examples

cost of predicting
example t

MDL probing: accuracy and complexity in one number

$$L_{\text{online}} = \sum_t -\log_2 p_{\theta_t}(y_t | h_t)$$

total bits paid across
the data stream

probe trained on the
first $t-1$ examples

cost of predicting
example t

Why it beats accuracy + selectivity

A memorising probe pays for its own complexity, early in the stream it compresses nothing. A representation that makes the concept easy yields short codes from the first examples. One number; no separate control task needed.

Amnesic probing: deleting the concept

$$h' = P_c^\perp \cdot h$$

Amnesic probing: deleting the concept

$$h' = P_c^\perp \cdot h$$

projects out the concept
subspace, INLP / LEACE

Amnesic probing: deleting the concept

$$h' = P_c^\perp \cdot h$$

projects out the concept
subspace, INLP / LEACE

then run the model
on h' , not the probe

Amnesic probing: deleting the concept

$$h' = P_c^\perp \cdot h$$

projects out the concept
subspace, INLP / LEACE

then run the model
on h' , not the probe

Behaviour changes

The concept was load-bearing at this layer.

Behaviour unchanged

Encoded, decodable, and ignored.

This is the experiment the causation-gap slide imagines. It has a name and a method.

Elazar et al. (2021), Amnesic Probing · Belrose et al. (2023), LEACE

Interchange interventions and causal abstraction

- 1 Hypothesise a high-level algorithm with variable V (e.g. “subject gender”).
- 2 Align V to a subspace of h , learned, not hand-picked (DAS: distributed alignment search).
- 3 Interchange: run x_1 , but splice in the V -subspace activations from x_2 .
- 4 Prediction: output changes as if the high-level variable were swapped, and nothing else.

$$\text{IIA} = \Pr[\text{model}(x_1 \parallel V \leftarrow x_2) = \text{algorithm}(x_1 \parallel V \leftarrow x_2)]$$

High interchange accuracy = the strongest available evidence that the representation is used, as hypothesised.

Superposition breaks linearity

$$\varepsilon = \max_{i \neq j} |w_i \cdot w_j|$$

Superposition breaks linearity

$$\epsilon = \max_{i \neq j} |w_i \cdot w_j|$$

interference,
the price of overlap

Superposition breaks linearity

$$\epsilon = \max_{i \neq j} |w_i \cdot w_j|$$

interference,
the price of overlap

overlap between two
feature directions

Superposition breaks linearity

$$\epsilon = \max_{i \neq j} |w_i \cdot w_j|$$

interference,
the price of overlap

overlap between two
feature directions

Johnson–Lindenstrauss: exponentially many near-orthogonal directions fit in $\mathbb{R}^d$, capacity bought by paying interference, so individual concepts are **not cleanly linearly separable**.

Linear probe fails →

Concept may be absent, nonlinearly encoded, in superposition, or distributed. Try a shallow MLP before concluding absence.

MLP probe succeeds →

Information present but not linearly organised. Mechanistic claim weaker, needs further investigation.

What probes cannot tell you

Causation	Accuracy doesn't show the model uses this for its outputs. That needs patching.
Readout vs encoding	An expressive probe's accuracy may be its own computation. Needs capacity limits, MDL, controls.
Mechanism	What is encoded, not how or where it was computed.
Completeness	You found one encoding. Others may exist; this may not be the one used.
Intent	Encoding harmful content is not intent to harm.
Generalisation	A probe fit on your dataset may not transfer to another distribution.

PART VII

Safety Applications

What representations reveal that outputs cannot

Probes in deployment

Result	What it showed
Probes in production , Kramár et al., DeepMind (2026)	Informed the deployment of cyber-misuse probes in user-facing Gemini. A probe + LLM cascade beat the LLM alone on test loss at roughly 1/50th its inference cost, calling the LLM under 10% of the time.
Sleeper-agent probes , MacDiarmid et al., Anthropic (2024)	Simple linear probes on generic contrast pairs detected backdoored “defection” at ~99% AUROC, without knowing the trigger. Research post, not peer-reviewed.
Deception probes , Goldowsky-Dill et al., Apollo (2025)	Linear probes flag strategic deception in agentic settings, and the signal precedes the deceptive output.
Refusal is a direction , Arditi et al. (2024)	Across the open chat models tested, refusal is mediated by a single direction: ablate it and refusal largely collapses; add it and the model refuses harmless prompts too.
Circuit breakers , Zou et al. (2024)	Remap harmful-state representations during generation. Reported more robust under red-teaming than output filtering.

A recurring pattern: the safety-relevant property tends to be low-dimensional, which is why a one-layer probe can do real work.

The safety gap

RLHF changes what a model outputs.

The safety gap

RLHF changes what a model outputs.

It does not necessarily erase what the model represents internally.

The safety gap

RLHF changes what a model outputs.

It does not necessarily erase what the model represents internally.

The risk

A safety-trained model may refuse harmful requests, but harmful knowledge may still be linearly decodable from mid-layer representations. Adversarial prompts can sometimes recover it.

The useful framing: probing is a second opinion. Test what the model represents, independently of what it says.

Sleeper agents and deceptive alignment

Sleeper Agents, Hubinger, Denison, Barez et al. (2024)

Models can be trained to behave safely during evaluation while concealing harmful behaviours for deployment. Safety training does not reliably remove deceptive behaviours.

Sycophancy to Subterfuge, Denison, MacDiarmid, Barez et al. (2024)

Probing reveals internal states that precede sycophantic and subterfuge behaviours before they manifest in outputs.

Why this pair of results matters

Together they show why output-level evaluation isn't enough: probing internal representations, not just outputs, can catch deceptive alignment before it ever shows up in behaviour.

Chain-of-Thought Is Not Explainability

“Chain-of-thought is a rationalisation produced after the decision is made, not a window into reasoning.”

What we found, Barez, Wu et al. (2025)

We add a biasing cue → the model flips its answer → it then rationalises the new answer in its CoT. The reasoning is post-hoc: probing shows the answer direction fires *before* the CoT is written.

Why this matters

If CoT is unfaithful, it can't serve as a safety monitor or as evidence of genuine reasoning. The internal signal turns out to be more reliable than the model's self-report.

Where probes break

Distribution shift, now measured, not hypothetical

Probes trained on short contexts fail to generalise to long-context inputs. A new architecture (MultiMax) improves long-context generalisation; training directly on long context costs over an order of magnitude more (Kramár et al., 2026).

Adaptive attacks are not solved

Once a probe gates behaviour, training pressure (or an adversary) optimises against it. The Gemini work is explicit that its techniques *do not* significantly reduce the success rate of adaptive adversarial attacks, and argues this may be extremely hard in general.

Monitoring the monitor

Probe fires → then what? Thresholds, escalation, and audit are governance questions. The dot product is the easy part.

Where this stands

Representation-level monitoring is becoming a real layer of the safety stack: it earns a place alongside RLHF and output classifiers, not in place of them. Defence in depth.

Dynamic safety monitoring

Beyond Linear Probes, Oldfield, [...], Barez

Static linear probes are a starting point, not a final solution. Adaptive, online monitoring systems that update as the model's representations shift during inference.

SafetyNet, Chaudhary, Barez

Detecting harmful LLM outputs by modelling and monitoring deceptive behaviours at the representation level, rather than classifying outputs after generation.

Where this is heading

From “what is encoded?” to “what do we do about it?”: probing pipelines that trigger auditing and governance decisions automatically. More at aigi.ox.ac.uk

PART VIII

Designing a Study

From question to defensible conclusion

Step 1: Choose the right probe type

Default

Always start with logistic regression. Strongest claims. Highest selectivity.

If linear fails

Try a shallow MLP ($d = 10-50$). Acknowledge the weaker claim. Never cite raw accuracy alone.

Avoid

Deep MLPs for interpretive claims. L2 regularisation does not improve selectivity, only capacity reduction does.

Step 2: Report all three baselines

Majority class

Accuracy from always predicting the modal class. The floor. Without this the result is uninterpretable.

Random repr.

Same probe on h from an untrained model. Controls for data statistics before any training.

Control task

Probe with shuffled (type-level) labels. Selectivity = linguistic – control.

Never report raw accuracy alone.

Steps 3 & 4: Layer profile and causal check

Step 3: Probe every layer

Never report only the best layer, that is p-hacking.

The layer profile is a finding. Where does accuracy peak? Does it drop in final layers? The shape tells you where the concept lives. Always show the full profile.

Step 4: Validate causally

If your claim requires causation you must run activation patching (or interchange interventions).

Probe accuracy alone is not sufficient, the most commonly violated principle in the probing literature. “The model uses sentiment” requires patching, not probing.

Reporting a probing result

“We trained a linear probe (logistic regression, $C = 0.1$) on [CLS] embeddings from all 12 layers of BERT-base to predict [concept].

The probe achieved $X\%$ linguistic accuracy (majority baseline $Y\%$, random representation baseline $Z\%$, control task accuracy $W\%$, selectivity $X - W\%$). Online codelength: X kbits (control: Y kbits).

These results indicate [concept] is linearly decodable from BERT layer L with selectivity $S\%$, suggesting it is encoded in the trained representations beyond what type-level memorisation explains.”

Language matters

“Linearly decodable”, not “the model knows”. “Suggesting”, not “proving”. Precision is not weakness; overclaiming erodes trust.

Summary

Seven things to remember

1. Freeze the encoder Always. Without this you are fine-tuning, not probing.

Seven things to remember

1. Freeze the encoder Always. Without this you are fine-tuning, not probing.

2. Linear probe first Supports the strongest claims. Justify any increase in complexity.

Seven things to remember

1. Freeze the encoder Always. Without this you are fine-tuning, not probing.

2. Linear probe first Supports the strongest claims. Justify any increase in complexity.

3. Linear $\neq$ nonlinear Direction in $\mathbb{R}^d$ vs decodable. Very different claims.

Seven things to remember

1. Freeze the encoder Always. Without this you are fine-tuning, not probing.

2. Linear probe first Supports the strongest claims. Justify any increase in complexity.

3. Linear $\neq$ nonlinear Direction in $\mathbb{R}^d$ vs decodable. Very different claims.

4. All three baselines Majority + random representations + selectivity. Never raw accuracy.

Seven things to remember

1. Freeze the encoder Always. Without this you are fine-tuning, not probing.

2. Linear probe first Supports the strongest claims. Justify any increase in complexity.

3. Linear $\neq$ nonlinear Direction in $\mathbb{R}^d$ vs decodable. Very different claims.

4. All three baselines Majority + random representations + selectivity. Never raw accuracy.

5. Probe every layer The layer profile is the finding. Show the full curve.

Seven things to remember

1. Freeze the encoder Always. Without this you are fine-tuning, not probing.

2. Linear probe first Supports the strongest claims. Justify any increase in complexity.

3. Linear $\neq$ nonlinear Direction in $\mathbb{R}^d$ vs decodable. Very different claims.

4. All three baselines Majority + random representations + selectivity. Never raw accuracy.

5. Probe every layer The layer profile is the finding. Show the full curve.

6. Accuracy $\neq$ causation Probes show accessibility. Patching shows use.

Seven things to remember

- 1. Freeze the encoder** Always. Without this you are fine-tuning, not probing.
- 2. Linear probe first** Supports the strongest claims. Justify any increase in complexity.
- 3. Linear $\neq$ nonlinear** Direction in $\mathbb{R}^d$ vs decodable. Very different claims.
- 4. All three baselines** Majority + random representations + selectivity. Never raw accuracy.
- 5. Probe every layer** The layer profile is the finding. Show the full curve.
- 6. Accuracy $\neq$ causation** Probes show accessibility. Patching shows use.
- 7. State your limits** What the result cannot conclude. Always explicit.

Key references

Alain & Bengio (2016) Linear Classifier Probes, the original framework.

Zhang & Bowman (2018) Random-encoder baseline.

Tenney et al. (2019) BERT Rediscovered the Classical NLP Pipeline.

Hewitt & Liang (2019) Control tasks, introduces selectivity.

Hewitt & Manning (2019) A Structural Probe for Finding Syntax.

Pimentel et al. (2020) Information-Theoretic Probing.

Xu et al. (2020) Usable Information Under Computational Constraints.

Voita & Titov (2020) MDL Probing.

Elazar et al. (2021) Amnesic Probing.

Geva et al. (2021) FFN Layers Are Key-Value Memories.

Geiger et al. (2021, 2023) Causal Abstraction · DAS.

Burns et al. (2022) Latent Knowledge Without Supervision.

Elhage et al. (2022) Toy Models of Superposition.

Zou et al. (2023) Representation Engineering.

Belrose et al. (2023) / Sharma et al. (2025) LEACE · Constitutional Classifiers.

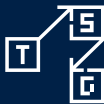
Barez et al. (2024–2025) · Kramár et al. (2026)
Sleepers Agents · CoT Is Not Explainability · Beyond Linear Probes · Context Matters · Production Probes for Gemini.

Discussion

“Probing gives us empirical windows into the black box, used with the right type, proper baselines, and honest scope.”

Discussion questions

- When would you choose an MLP probe over a linear probe, and what exactly do you give up?
- Can you design a probe for misaligned goal representations? What would the control task look like?
- Why does reducing probe dimension improve selectivity more than L2 regularisation? (Hint: $\mathcal{V}$ -information.)
- An unconstrained probe recovers the concept from a randomly initialised encoder. What, then, did probing the trained encoder ever tell you, and what assumption rescues the method?



TECHNICAL
SAFETY &
GOVERNANCE
LAB

OxML Summer School
2026

Thank you



★ I'm hiring 2 Research Assistants.

Interpretability & AI safety. Postdocs, PhD students, and interns too. Come talk to me.

✕ @FazlBarez | 🌐 fazlbarez.com